VirtioFS in IncludeOS

University of Oslo

Torgeir Skovholt

April 20, 2026

Abstract

This thesis presents the design, implementation, and performance evaluation of a VirtioFS file-system driver for IncludeOS, a unikernel operating system. VirtioFS enables a Linux host to share a directory with a guest virtual machine over the FUSE protocol transported via Virtio virtqueues. Implementing VirtioFS in IncludeOS required a redesign of the existing virtual filesystem (VFS) architecture to support clean integration of alternative file-system drivers, followed by implementation of a Virtio PCI transport layer and a VirtioFS device driver.

Controlled microbenchmarks comparing the IncludeOS VirtioFS driver against a Linux guest running under the same QEMU/KVM infrastructure show that IncludeOS achieves substantially higher I/O throughput across all access patterns. In polling mode, peak sequential read throughput reaches 4,786 MiB/s — a $\times 1.74$ speedup over Linux POSIX — with up to $\times 3.02$ at small 8 KiB chunk sizes where per-request overhead dominates. An investigation of io_uring SQPOLL as a Linux-side optimisation reveals a counterintuitive regression of 9–38% relative to plain POSIX I/O.

An end-to-end video decoding benchmark using the C63 educational codec shows that the I/O throughput advantage translates into a 22.7% reduction in wall-clock decode time, and eliminates the 25.3% cold-start penalty observed on Linux. These results validate the hypothesis that removing general-purpose OS abstractions — system calls, the VFS, FUSE framing, and a user-space daemon — yields measurable I/O performance improvements that propagate to realistic application workloads.

Contents

1 Motivation **2**

1.1 Research Questions . . . 3

1.1.1 Primary Hypothesis . . . 3

1.1.2 Extended Hypothesis . . . 3

1.2 Method . . . 4

2 Background **5**

2.1 System Emulation and Virtualization . . . 5

2.1.1 Overview . . . 5

2.1.2 QEMU and KVM . . . 5

2.1.3 Vhost-User Architecture . . . 6

2.2 VirtioFS . . . 6

2.3 Dual Ring Buffer . . . 6

2.4 Atomics . . . 8

2.5 Memory Ordering . . . 8

2.5.1 The x86 Memory Model . . . 8

2.5.2 The C++ Memory Model . . . 8

2.6 io_uring . . . 8

2.7 Virtio Standard . . . 9

2.7.1 Introduction . . . 9

2.7.2 Device Discovery via PCI . . . 10

2.7.3 Fences are complicated . . . 12

2.8 FUSE . . . 12

2.8.1 Protocol . . . 12

2.8.2 Initialization Handshake . . . 12

2.8.3 File Operations . . . 12

2.9 IncludeOS . . . 13

2.9.1 Overview . . . 13

2.9.2 PCI Interface . . . 13

2.9.3 Events and Interrupts . . . 13

2.9.4 Filesystem Support and MUSL Integration 14

3 Design **15**
3.1 Virtual Filesystem Layer 15
3.2 Virtio PCI Layer 16
3.3 VirtioFS Device 16
3.3.1 Initialization Sequence 18
3.3.2 Synchronous Polling Model 18
3.3.3 Combined Interaction 18

4 Implementation **21**
4.1 Virtual Filesystem Layer 21
4.1.1 sys_open 21
4.1.2 sys_read / sys_readv / sys_write / sys_writev 22
4.1.3 sys_close 22
4.1.4 sys_unlink 23
4.2 Virtio PCI Layer 24
4.3 VirtioFS Device Driver 26
4.3.1 Initialisation 26
4.3.2 Inode Resolution 26
4.3.3 Open, Create, and Close 27
4.3.4 Read and Write 27
4.3.5 Seek, Unlink, and Registration 28

5 Evaluation **29**
5.1 Experimental Setup 29
5.1.1 Virtual-Machine Infrastructure 29
5.1.2 Timing 31
5.1.3 Reproducible Builds 31
5.1.4 Host machine 31
5.1.5 POSIX I/O Benchmark (Experiments 1 & 2) 31
5.1.6 io_uring Benchmark (Experiment 3) 32
5.1.7 C63 Video Decoder Benchmark 32
5.2 RQ1 – VirtioFS Throughput: IncludeOS vs. Linux 33
5.2.1 Results 33
5.2.2 Analysis 35
5.3 RQ2 – Tweaking Linux: io_uring SQPOLL 35
5.3.1 Results 36

5.3.2 Analysis 39
5.4 RQ3 – Impact on Video Decoding 39
5.4.1 Results 39
5.4.2 Analysis 40
5.5 Threats to Validity 41

6 Conclusion and Future Work **42**
6.1 Future Work 43

Chapter 1

Motivation

The project started with a project listing issued by my supervisor regarding implementing a Virtio file system (VirtioFS) driver in IncludeOS, a unikernel operating system. Unikernels are libraries that can be linked with application code forming an application capable of running directly on bare metal. A unikernel executes entirely in kernel mode, thereby eliminating context switch overhead present in operating systems such as Linux. This realization motivated an investigation into whether a VirtioFS driver implemented in IncludeOS could achieve higher performance than its Linux counterpart, particularly with respect to IO throughput.

File system support in IncludeOS comprises a virtual file system (VFS) layer, file descriptor abstractions, incomplete FAT16 and FAT32 drivers, and Virtio and memory disk block drivers. The VFS layer is tightly coupled with FAT, hindering clean integrations of alternative file systems such as VirtioFS. Therefore, a redesign of the VFS architecture is necessary to enable clean integration of alternative file systems. Developing block-level file system drivers is complex and prone to errors. A VirtioFS implementation addresses this by delegating file system operations to a file system server that performs system calls through the host kernel, thereby leveraging the host kernel's existing file system driver implementations.

Our findings showed that IncludeOS outperformed the Linux guest in file system throughput. However, during this work we discovered io_uring, a modern Linux interface that reduces system call overhead by using shared ring buffers between user and kernel space. io_uring can optionally support polling instead of interrupts, providing fine-grained control over I/O execution. These characteristics indicate that io_uring could potentially close the performance gap with IncludeOS, motivating a further investigation into whether it could rival the throughput achieved

by the unikernel approach.

After a brief detour to evaluate io_uring as an alternative interface, we return to assessing the driver through an application. To assess the impact of file system performance on applications, this project examines video decoding using the C63 toy codec. Video decoding is I/O intensive and bug-prone, making it well suited for isolated execution in IncludeOS virtual machines under QEMU and KVM, similar to microvirtualization approaches such as Bromium.

1.1 Research Questions

- **RQ1:** Can a VirtioFS implementation in IncludeOS have a higher I/O throughput than Linux?
- **RQ2:** Why is the I/O throughput in VirtioFS Linux slower than IncludeOS? Can we tweak Linux to achieve equivalent performance?
- **RQ3:** What impact will enhanced I/O throughput have on common tasks such as video decoding?

1.1.1 Primary Hypothesis

We hypothesize that an IncludeOS implementation will rival Linux in POSIX performance measurements. The explanation for this hypothesis is twofold. IncludeOS does not preempt by design and contains less overall code. Therefore, IncludeOS has lower overhead with better CPU efficiency. Furthermore, IncludeOS operates without the assumption of kernel and user space isolation, thus allowing for a multitude of zero-copy techniques that would not be tolerated in a general-purpose operating system such as Linux.

1.1.2 Extended Hypothesis

Our findings confirm the main hypothesis. Through hands-on experimentation with the problem domain, we developed a deeper, more nuanced understanding of I/O performance bottlenecks. To address this, we examined io_uring, a proposed Linux kernel interface designed to reduce syscall overhead by batching I/O operations or a kernel. io_uring enables developers to optimize I/O performance through several mechanisms: fine-tuning device driver behaviour, optimizing data movement patterns, and eliminating unnecessary system calls.

1.2 Method

The thesis employs standard scientific hypothesis formulation about performance metrics and driver prototyping. The prototyping step includes reading available device specifications and common behaviour for device emulation, utilizing reverse engineering methods whenever documentation is lacking. System thinking and inference hypothesis for device behaviour will be confirmed with dynamic and static analysis.

Chapter 2

Background

2.1 System Emulation and Virtualization

2.1.1 Overview

System emulation involves reproducing the behaviour of a complete machine in software, including its CPU, memory subsystem, chipset, and peripheral devices. Each component is implemented as a separate subsystem within the emulator. The CPU emulator executes instructions until an operation requires interaction with an emulated device, at which point control transfers to the appropriate device emulation subsystem. This model is known as **trap-and-emulate**, rooted in the foundational Popek and Goldberg [1] virtualization paper, which establishes that sensitive instructions must trap so they can be intercepted and handled by the virtual machine monitor (VMM).

2.1.2 QEMU and KVM

QEMU [2] is an open-source machine emulator supporting full system emulation across multiple CPU architectures. In pure software mode it uses dynamic binary translation via its Tiny Code Generator (TCG). When hardware virtualization extensions are available — Intel VT-x or AMD SVM — QEMU delegates CPU execution to **KVM** [3], a Linux kernel module that exposes an API for building hypervisors. Under KVM, guest instructions execute directly on the host CPU; only sensitive instructions cause VM exits.

QEMU interacts with KVM through `ioctl` calls on `/dev/kvm`. vCPUs are managed as ordinary Linux threads via `KVM_RUN`, which blocks until the vCPU hits a sensitive

instruction or accesses an MMIO/PMIO region. For higher-performance signalling, KVM supports `ioeventfd` [4]: event file descriptors that deliver guest-write notifications directly to a waiting user-space thread, bypassing the QEMU event loop entirely. This mechanism underpins vhost-user and VirtioFSD.

2.1.3 Vhost-User Architecture

QEMU supports offloading device emulation to auxiliary user-space daemons via the **vhost-user** [5] protocol, sharing the following over a UNIX domain socket:

- Physical addresses of Virtio queue memory regions — enabling direct shared-memory access without copying through QEMU.
- `ioeventfd` descriptors for kick notification.
- `irqfd` descriptors for interrupt injection into the guest.

VirtioFSD is the principal example, implementing a complete file-system device in a separate user-space daemon, handling the full request lifecycle — notification, parsing, processing, and response — without involving QEMU.

2.2 VirtioFS

VirtioFS [**?**] enables a Linux host and its guest VM to share a designated host directory. The guest runs the VirtioFS kernel driver; on the host **VirtioFSD** acts as a remote file-system server. Communication uses the FUSE protocol transported over Virtio virtqueues. The shared memory is accessible to both sides without the need for copying.

Queue 0 is the *high-priority* queue for latency-sensitive messages such as `FUSE_FORGET`; queue 1 is the general-purpose *request* queue. VirtioFS supports scatter-gather I/O: a single descriptor chain may reference multiple discontiguous guest buffers.

2.3 Dual Ring Buffer

The circular ring buffer is one of the most ubiquitous data structures in systems programming. It enables lockless access by having one thread enqueue entries while another dequeues and consumes them — as long as only one producer and one consumer operate concurrently, no locks are required.

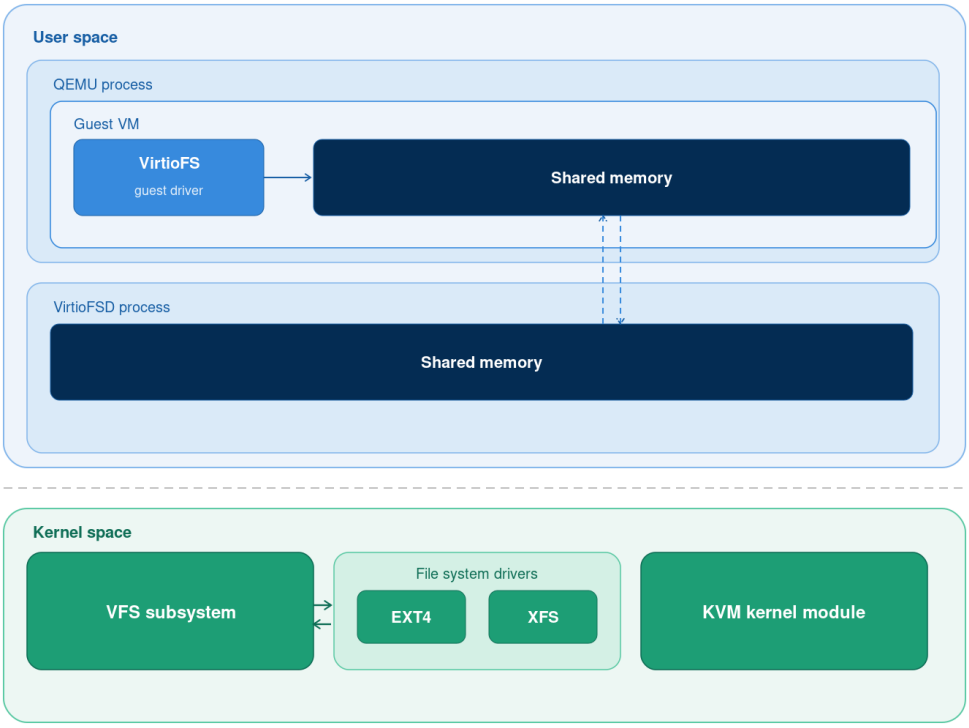

Figure 2.1: VirtioFS static component overview

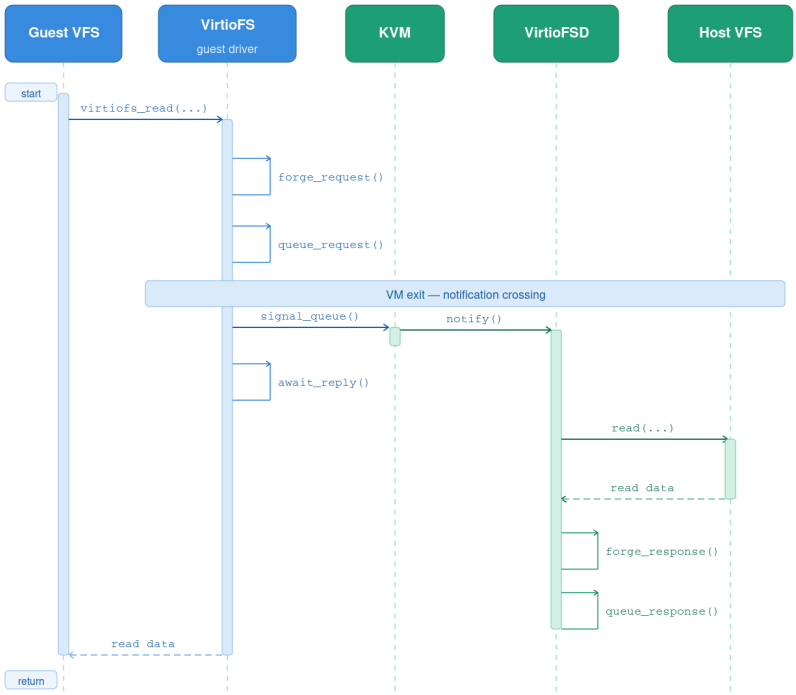

Figure 2.2: An abstract sequence diagram modeling a guest read through the VirtioFS stack. Blue = guest-side actors; teal = host-side.

The **dual ring buffer** pattern consists of two circular ring buffers. The first ring buffer carries requests from client to server; the second carries responses from server to client. Together they form a lockless request–response channel. This abstract pattern is the foundation for both io_uring and Virtio.

2.4 Atomics

Compiler atomics and CPU atomics

2.5 Memory Ordering

2.5.1 The x86 Memory Model

The x86 architecture implements a relatively strong memory model known as **Total Store Ordering (TSO)**. Under TSO, read and store instructions are generally not reordered with one notable exception: older stores may be reordered with younger loads to *different* memory locations. Critically, the executing CPU itself still sees a consistent view of its own memory operations — the reordering is only visible to *other* CPUs observing the memory bus.

2.5.2 The C++ Memory Model

The C++ memory model, introduced in C++11, provides a portable abstraction over hardware memory models. It defines a set of memory orderings that the programmer can attach to atomic operations to control visibility and synchronization guarantees across threads:

In the context of the Virtio split queue, the C++ memory model is applied precisely: a `memory_order_release` fence is issued after populating the descriptor chain and before incrementing the available ring index, ensuring all descriptor writes are visible to the device before it observes the updated index. A subsequent `memory_order_seq_cst` fence precedes the check of the used ring's notification suppression flag, preventing reordering that could cause a missed notification.

2.6 io_uring

io_uring is a modern Linux I/O interface introduced in kernel 5.1, designed to address the performance inefficiencies of POSIX I/O and earlier asynchronous I/O

mechanisms such as AIO. The primary selling point of io_uring is its ability to greatly reduce — and in some configurations eliminate entirely — the number of system calls required to perform I/O.

io_uring is built on the dual ring buffer pattern described in Section 2.3. It exposes two shared-memory ring buffers between user space and the kernel: the **submission queue (SQ)**, into which the application places I/O requests, and the **completion queue (CQ)**, from which the application reads completions. Because both rings live in shared memory, the application can submit requests and harvest completions without any system call, provided the kernel is actively polling the submission queue.

io_uring supports a polling mode called **SQPOLL**, in which a dedicated kernel thread continuously polls the submission queue for new entries. In this mode, applications can perform I/O with zero system calls for the duration of the polling window. When the kernel thread has been idle for a configurable timeout, it sleeps and requires a wakeup syscall (`io_uring_enter`) to resume.

liburing is a user-space wrapper library around the io_uring API. io_uring's raw interface involves careful management of ring indices, memory barriers, and kernel flags; liburing encapsulates this complexity and provides a higher-level interface that substantially reduces the boilerplate required when writing io_uring applications.

The architectural similarity between io_uring and Virtio is not coincidental: both implement the dual ring buffer pattern over shared memory to decouple request submission from completion processing, enabling high-throughput asynchronous I/O without per-operation kernel crossings.

2.7 Virtio Standard

2.7.1 Introduction

Virtio [**?**] is a paravirtualization standard for efficient data exchange between a guest *driver* and a host *device*. It separates a **control plane** — device discovery, configuration, and feature negotiation — from a **data plane** that exchanges bulk data via shared-memory ring structures called **virtqueues**

2.7.2 Device Discovery via PCI

Virtio devices advertise their register regions through vendor-specific PCI capabilities, each with a `cfg_type` identifying the region and `bar`, `offset`, and `length` fields locating it. Four standard Virtio capability types are defined:

- `VIRTIO_PCI_CAP_COMMON_CFG`: device-wide and per-queue configuration registers.
- `VIRTIO_PCI_CAP_NOTIFY_CFG`: queue notification region.
- `VIRTIO_PCI_CAP_ISR_CFG`: interrupt status register.
- `VIRTIO_PCI_CAP_DEVICE_CFG`: device-class-specific registers.

The control plane governs the full device lifecycle. The specification prescribes strict ordering: reset (`device_status = 0`), set `ACKNOWLEDGE` and `DRIVER` bits, negotiate features, set `FEATURES_OK` and verify the device retains it, configure queues, then set `DRIVER_OK`. After setup, MMIO is used only for queue kicks; under KVM each kick delivers an `ioeventfd` notification directly to the device thread.

Descriptor Area

The descriptor area is an array of descriptor entries each carrying a guest-physical **Buffer** address, a byte **Len**, **Flags**, and a chain **Next** index. The **Next** is valid when the `NEXT` flag is set in the same entry. A writeable descriptor is distinguished from a readable descriptor by having `WRITE` flag. Readable descriptors must precede all writable descriptors — no interleaving is permitted.

Available and Used Areas

The driver and device communicate using two areas with similar structure: Available and used area. The areas **MUST** only be modified by one end and read by the other. The driver modifies the available area while the device modifies the used area. The driver reads the used area while the device reads the available area. The areas follow the dual ring buffer pattern described in section 2.3 with additional flag fields for signaling that notifications are not needed for processing the request.

Request Submission — Driver to Device

The driver allocates free descriptors, populates them, links them via the `next` field, places the head index into the available ring's `ring[]`, and increments the available ring's `idx` to expose the request. A `memory_order_release` fence precedes the counter advance. The driver then conditionally writes the queue index to the notification register to kick the device. The tracks request by comparing `_avail_ring->idx` against its locally maintained `_last_avail_idx`.

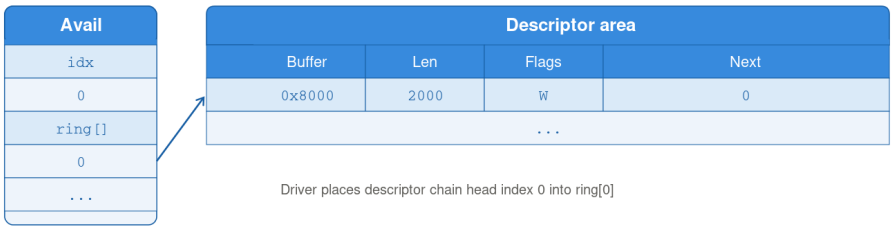

Figure 2.3: Driver places descriptor chain head index 0 into `ring[0]`.

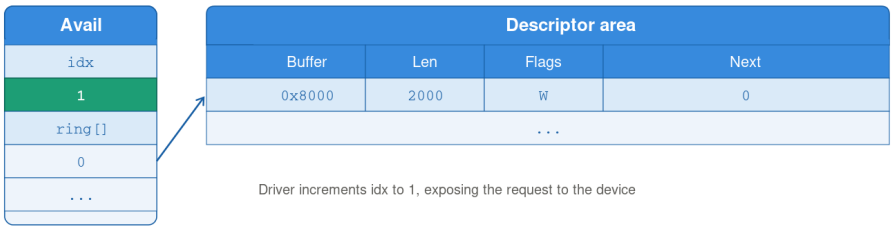

Figure 2.4: Driver increments `idx` to 1 (green), exposing the request.

Request Completion — Device to Driver

After processing a request, the device writes a `virtq_used_elem` — containing the head descriptor index and bytes written into writable buffers — and increments the used ring's `idx`. A store fence **MUST** precede the increment 2.5. The driver detects completion by comparing `_used_ring->idx` against its locally maintained `last_used_idx`.

Notification Mechanisms

Both sides can suppress notifications. The driver sets `VIRTQ_AVAIL_F_NO_INTERRUPT` to inhibit device interrupts; the device sets `VIRTQ_USED_F_NO_NOTIFY` to inhibit driver kicks. These flags enable runtime switching between interrupt-driven and polling modes.

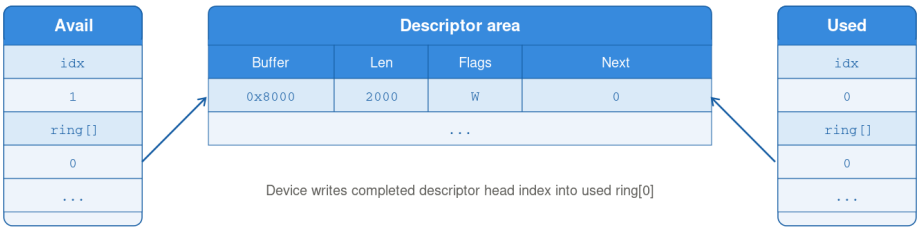

Figure 2.5: Device writes the completed descriptor head index into used `ring[0]`.

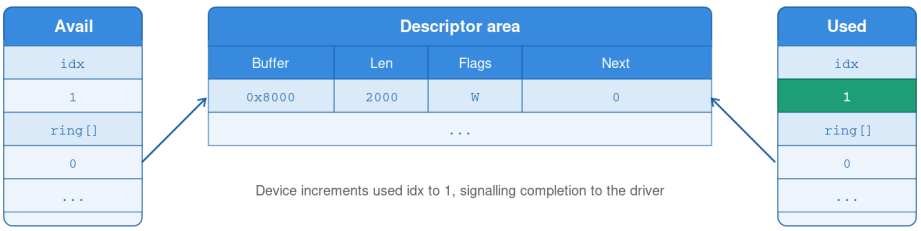

Figure 2.6: Device increments used `idx` to 1 (green), signalling completion.

2.7.3 Fences are complicated

2.8 FUSE

2.8.1 Protocol

FUSE [?] exports file-system operations to a user-space server. Every request begins with `fuse_in_header` carrying the opcode, total length, a monotonically increasing `unique` identifier, and the target inode number. Every response begins with `fuse_out_header` echoing `unique` and an error code.

2.8.2 Initialization Handshake

Client and server negotiate protocol version via `FUSE_INIT`. The client advertises its major and minor versions; the negotiated minor is the minimum of the two. If the server's major is lower, the client must re-send adjusted to the server's major. The response carries `max_write` — the maximum payload the server accepts per write. The client must cache and enforce this limit.

2.8.3 File Operations

A complete read sequence issues: `FUSE_LOOKUP` (path → inode), `FUSE_OPEN` (inode → 64-bit file handle), `FUSE_READ` (file handle, buffer, offset, length), `FUSE_RELEASE`

(free file handle), and `FUSE_FORGET` (decrement inode reference count). Writes follow an analogous pattern; `FUSE_CREATE` combines lookup and open for new files.

2.9 IncludeOS

2.9.1 Overview

IncludeOS [**?**] is a lightweight, bootable operating system implemented in C, C++, and assembly. Unlike conventional operating systems, IncludeOS follows the **unikernel** paradigm: it packages an application together with only the essential libraries and device drivers required for execution. The result is a single-address-space image with no process isolation, no user/kernel boundary, and no general-purpose scheduler. This design enables substantial reductions in both CPU and memory consumption compared to traditional virtual machines. The virtual address space is configured with identity mapping to physical memory for the first 512 GiB.

2.9.2 PCI Interface

Driver implementations receive a reference to a generic PCI device abstraction during construction. This abstraction exposes methods for PCI configuration space access (`read16`, `read32`), capability discovery (`probe_resources`, `parse_capabilities`), and MSI-X management (`msix_cap`, `init_msix`, `get_msix_vectors`). Before utilizing MSI-X functionality, drivers must invoke `probe_resources()` and `parse_capabilities()` to enumerate available resources. Following these preparatory calls, `init_msix()` configures MSI-X support, and `get_msix_vectors()` returns the number of available interrupt vectors. MSI-X vector identifiers are valid in the range zero to vector count exclusive.

2.9.3 Events and Interrupts

IncludeOS implements interrupt handling through an event-driven architecture based on callback queues. When a hardware interrupt fires, the system executes the delegate registered for the corresponding event number. A driver registers a delegate with `Events::get().subscribe()`, receiving an event number. This event number is then configured as the target for a specific interrupt vector via `setup_msix_vector()`. When the configured interrupt fires, the registered delegate executes asynchronously.

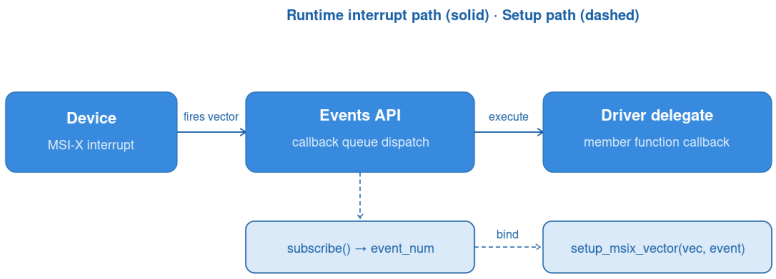

Figure 2.7: IncludeOS interrupt handling. Setup path (dashed): the driver subscribes a delegate to get an event number, then binds it to an MSI-X vector. Runtime path (solid): the device fires the vector, the Events API dispatches to the registered delegate.

2.9.4 Filesystem Support and MUSL Integration

IncludeOS provides filesystem functionality through two complementary approaches: a POSIX-compatible interface via the MUSL C standard library [**?**], and native IncludeOS filesystem abstractions. The filesystem implementation includes standard file descriptor abstractions for both generic and filesystem-specific descriptors. Filesystem paths are represented as tokenized strings using the forward slash as a separator.

IncludeOS integrates MUSL, a minimal implementation of the C standard library widely adopted in embedded systems development, distinguished by its low memory overhead. IncludeOS employs a patched version of MUSL that redirects standard system calls to external functions, enabling customization of system call behaviour. This patching mechanism, implemented in `src/musl/*.cpp`, allows IncludeOS to intercept and reimplement standard library system calls to align with the unikernel execution model — making it possible for applications to use VirtioFS transparently through standard POSIX file I/O without any application-level awareness of the underlying Virtio transport.

Chapter 3

Design

The VirtioFS driver is structured as three cooperating layers: a virtual filesystem (VFS) abstraction layer interfacing with the POSIX system-call layer; a Virtio PCI transport layer managing device initialization and queue operations; and a VirtioFS device layer composing the two lower layers into a complete VirtioFS.

3.1 Virtual Filesystem Layer

The VFS layer is deliberately thin: it performs path-based dispatch and file-descriptor bookkeeping, delegating all actual I/O to the registered driver. Four cooperating components form the layer:

- `fs::VFS` — singleton mount table mapping string prefixes to `fs::Filesystem` structs of function pointers (`open`, `read`, `readv`, `write`, `writev`, `lseek`, `close`, `unlink`).
- `FD_map` — file-descriptor allocator issuing integer descriptors mapped to polymorphic `FD` objects.
- `File_FD` — concrete file descriptor forwarding each method call with the descriptor integer to the owning driver's function pointer.
- Syscall stubs — outermost layer performing only path decomposition, returning standard `errno` values on failure.

A path such as `VirtioFS0/data/file.txt` is split at the first separator: the leading component selects the driver; the tail is forwarded. Writes to descriptors 1 and 2 are intercepted before any `FD_map` lookup and routed to `os::print`.

3.2 Virtio PCI Layer

The Virtio PCI layer is divided into two independent components reflecting the specification's separation of control-plane initialization and data-plane I/O. Keeping them separate allows the future implementation queue implementations such as packed queues.

`Virtio_control` encapsulates the full initialization sequence — device validation, capability discovery, reset, status-bit sequencing, and feature negotiation — completing before returning. Any failure triggers `_virtio_panic`, setting `VIRTIO_CONFIG_S_FAILED` and halting via `os::panic`. `VIRTIO_F_VERSION_1` is unconditionally required; optional features are accepted only if the device advertises them.

Virtio_control required and optional features are passed by the constructor down to the _negotiate_features (called by the constructor) method. Virtio_control an optional method for enabling MSI-X interrupts for deferred queue callback routines. Add a flowchart here!

Virtio_control exposes that is required by every device to call for finishing the Virtio PCI plane initialization sequence. That function is called set_driver_ok_bit. This function is called after the Virtio_control constructor, virtqueue constructors. Afterwards, the virtqueues are functional and operating.

`Split_queue` implements the split virtqueue, exposing direct control over descriptor chain composition for arbitrary scatter-gather I/O patterns. At construction it validates `queue_size`, allocates the three ring structures with mandated alignments, registers their physical addresses, and computes the per-queue notification address. A free-descriptor pool provides O(1) allocation and reclamation with no dynamic memory allocation on the I/O path.

3.3 VirtioFS Device

The VirtioFS device driver composes one `Virtio_control` and two `Split_queue` instances into a POSIX filesystem interface. Queue 0 is the high-priority queue; queue 1 the request queue. Both are configured in polling mode (`VIRTIO_MSI_NO_VECTOR`).

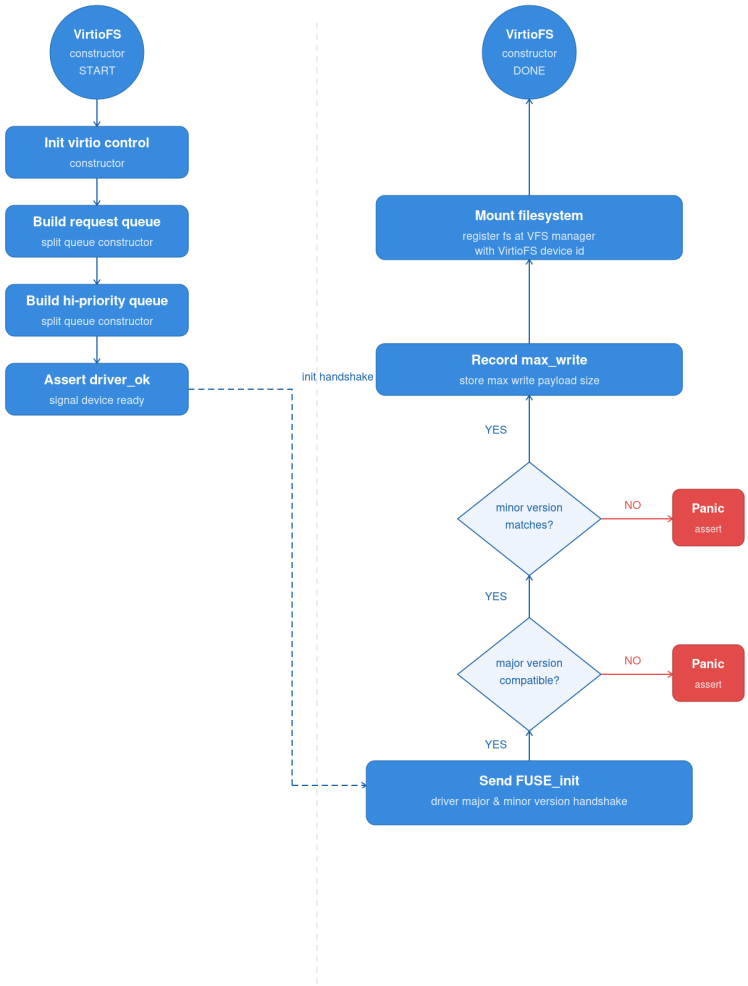

Figure 3.1: VirtioFS constructor initialization sequence. Left (top-down): Virtio setup. Right (bottom-up): FUSE handshake, version checks, `max_write` record, filesystem mount.

3.3.1 Initialization Sequence

Initialization proceeds in three sequential phases. First, `Virtio_control` completes PCI negotiation, both queues are registered, and `set_driver_ok_bit` signals readiness. Second, the `FUSE_INIT` handshake advertises major version 7 and minor version 36; version numbers are validated and `max_write` is stored, reduced by `FUSE_BUFFER_HEADER_SIZE`. Third, an `fs::Filesystem` struct is registered with `fs::VFS` under `VirtioFS{id}`.

3.3.2 Synchronous Polling Model

All filesystem operations are *synchronous and blocking*: at most one request is in flight at any time, and the CPU spins on `has_processed_used` until completion. Busy-waiting in this manner eliminates scheduling latency and maximizes I/O throughput. Request headers are allocated on the stack; caller-supplied buffers are enqueued directly as Virtio descriptor tokens, delivering data between caller memory and VirtioFSD without any intermediate copies.

3.3.3 Combined Interaction

Figure 3.2 shows the full lifetime of the three cooperating components across initialization and a single I/O operation.

During initialization `VirtioFS` first constructs `Virtio_control`, which drives the full PCI negotiation sequence — reset, feature negotiation, and `FEATURES_OK` — before returning. Both `Split_queue` instances are then constructed, each selecting its queue slot, allocating the three ring structures with mandated alignments, registering their physical addresses, and enabling the queue. A final call to `set_driver_ok_bit` signals device readiness and concludes the control-plane setup.

On the I/O path, `VirtioFS` calls `enqueue` on the appropriate `Split_queue` with a `VirtTokens` vector. `Split_queue` builds the descriptor chain, issues a release fence to guarantee descriptor visibility, increments the available ring index, and calls `_notify_device` to write the queue index to the pre-computed notification register address. `VirtioFS` then busy-waits on `has_processed_used` until the device writes a completion entry into the used ring, after which `dequeue` reclaims the descriptors and returns the `VirtTokens` to the caller.

Figure 3.3 shows the same lifecycle with MSI-X interrupts substituted for busy-polling. The initialization phase extends with three additional steps: `enable_msix`

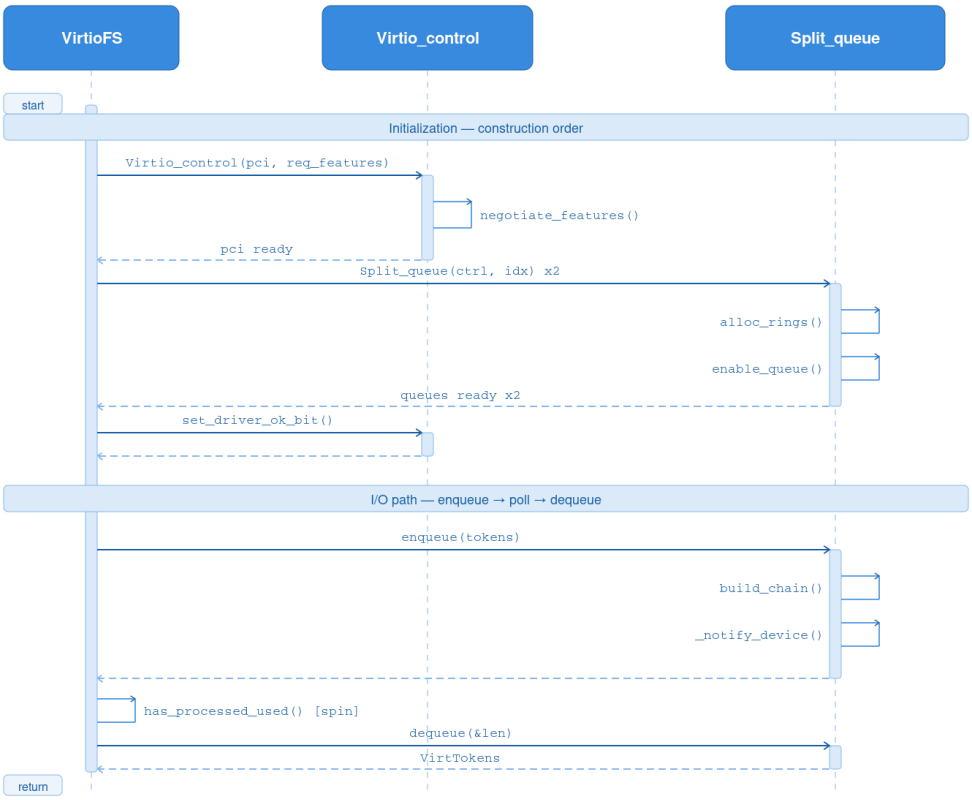

Figure 3.2: Sequence diagram of the combined interaction between `VirtioFS`, `Virtio_control`, and `Split_queue`. The initialization phase (top) establishes the Virtio PCI control plane and activates both queues before signalling driver readiness. The I/O path (bottom) shows a single request cycle: descriptor chain assembly, device notification, synchronous busy-wait, and dequeue.

configures the PCI MSI-X capability and returns the available vector count; `Events::subscribe` registers a completion callback in the IncludeOS event system and returns an event number; and `setup_msix_vector` binds that interrupt vector to the request queue. On the I/O path the spin loop is replaced by a `HLT` instruction that suspends the vCPU until the device fires a MSI-X interrupt, which the hypervisor delivers as a virtual interrupt and IncludeOS dispatches to the registered callback. Interrupt-driven completion frees the virtual processor for other work at the cost of additional context-switch latency compared with the polling model.

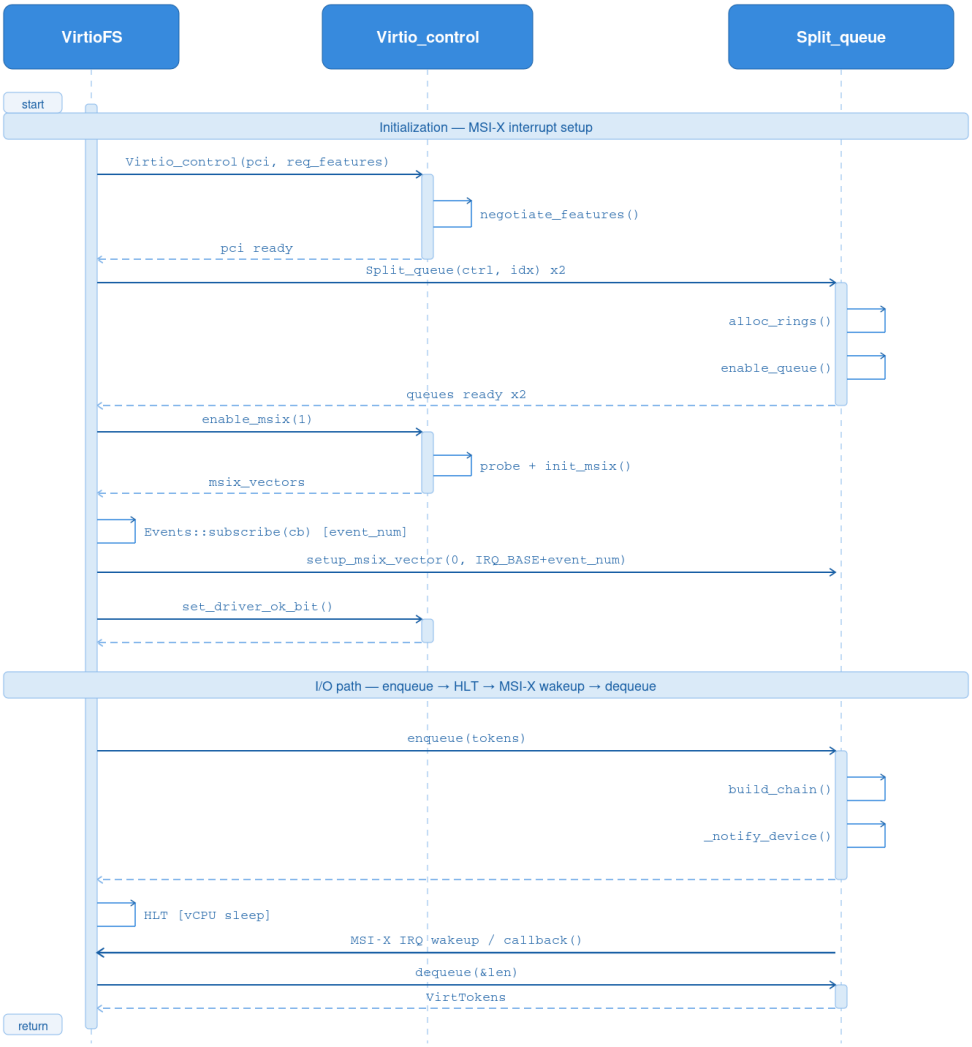

Figure 3.3: Sequence diagram of the interrupt-driven (MSI-X) variant. Compared with Figure 3.2, the initialization phase adds `enable_msix`, `Events::subscribe`, and `setup_msix_vector` calls; the busy-poll spin is replaced by a `HLT` sleep and a `MSI-X IRQ` wakeup arrow from the device.

Chapter 4

Implementation

4.1 Virtual Filesystem Layer

The VFS layer sits between the MUSL system call stubs and the registered filesystem driver. It is responsible for exactly two concerns: path-based mount dispatch during `open`, and file-descriptor-based dispatch for all subsequent I/O. Four syscall groups emerge from the implementation, each with a clearly defined pattern.

4.1.1 sys_open

`sys_open` is the only syscall that touches the mount table. The raw pathname is parsed into an `fs::Path` object, which tokenizes the string on the forward slash separator. The leading component — the mount prefix — is extracted via `path.front()` and looked up in `fs::VFS::get_mounts()`. If no matching mount exists, `-ENOENT` is returned immediately.

If the prefix resolves, the tail path is reconstructed via `path.pop_front()` and `path.to_string()` with a trailing separator stripped. A `File_FD` object is then allocated from `FD_map::_open<File_FD>(fs)`, which binds the file descriptor to the resolved `fs::Filesystem` struct at allocation time — this binding is the key structural decision: all subsequent I/O on this descriptor will dispatch through the function pointer table captured here, with no further mount table involvement. The integer descriptor is retrieved via `fde.get_id()` and passed — along with the tail path, flags, and mode — to the driver's `open` function pointer.

On success (result == 0) the integer descriptor is returned to the caller. On any

failure the descriptor must be explicitly released via `FD_map::close(fd)` to avoid leaking it. The catch block distinguishes two failure modes by inspecting whether `fd` is still zero: if so, the `FD_map` allocation itself failed (`-ENOENT`); otherwise the driver's open function threw (`-ENOSYS`).

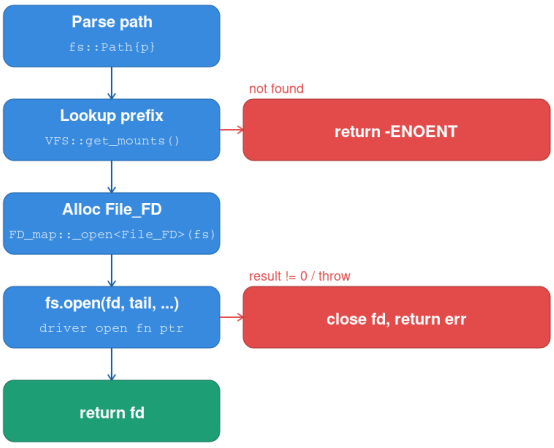

Figure 4.1: `sys_open`: path parse, prefix lookup, `File_FD` allocation, and driver call.

4.1.2 sys_read / sys_readv / sys_write / sys_writev

These four syscalls share a common structural pattern. Each follows the same three-step skeleton: call `FD_map::_get(fd)` to retrieve the `File_FD*` associated with the integer descriptor; if non-null, invoke the corresponding virtual method on the `File_FD`, which forwards through the `fs::Filesystem` function pointer to the driver; return `-EBADF` if the pointer is null or `-ENOSYS` if the driver throws. The VFS layer itself contains no I/O logic — all behaviour is entirely delegated.

`sys_write` and `sys_writev` diverge from this pattern for file descriptors 1 (stdout) and 2 (stderr). Both check for these reserved values *before* any `FD_map::_get` call and route output directly to `os::print()`, bypassing the VFS entirely. For `sys_writev`, each `iovec` element is printed individually in a loop. This intercept allows the C standard library output functions to work before any filesystem driver is registered, and without driver dispatch overhead.

4.1.3 sys_close

`sys_close` follows the same `FD_map::_get` lookup as the I/O group, but diverges in two important ways. First, it checks explicitly for a null `File_FD*` and returns

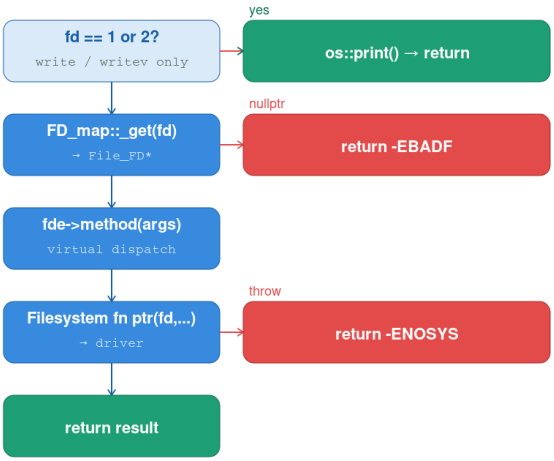

Figure 4.2: `sys_read/readv/write/writev`: stdout/stderr intercept (write/writev only), then shared fd dispatch through `FD_map`, virtual method, and driver function pointer.

`-EBADF` rather than relying on the try-catch path. Second, a non-zero return from the driver's `close` method is treated as unrecoverable: `sys_close` calls `os::panic()` with a diagnostic message rather than propagating the error upward. The fd is released via `FD_map::close(fd)` only after a successful driver close — ensuring the descriptor is not reissued while the driver still holds resources associated with it.

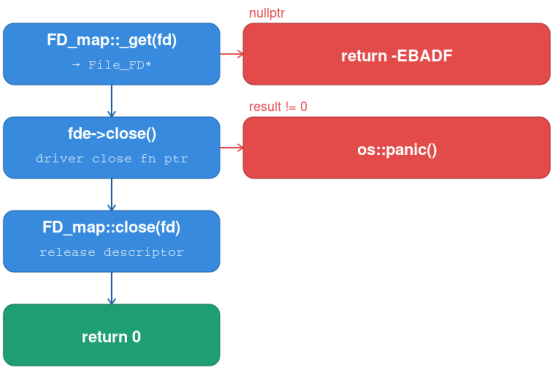

Figure 4.3: `sys_close`: explicit null check, driver close with panic on failure, then descriptor release.

4.1.4 sys_unlink

`sys_unlink` follows the same prefix-extraction pattern as `sys_open` — parse, `front()`, `contains()`, `pop_front()`, `to_string()`, strip — but skips fd allocation entirely and calls the driver's `unlink` function pointer directly with the tail path.

It uses a `submount_exists` boolean flag to distinguish between a missing mount (`-ENOSYS`) and a driver-thrown error (`-ENOENT`) in the catch block. `sys_unlink` is therefore the simplest syscall in the group: no fd lifecycle, no stdout intercept, just mount resolution and a direct driver call.

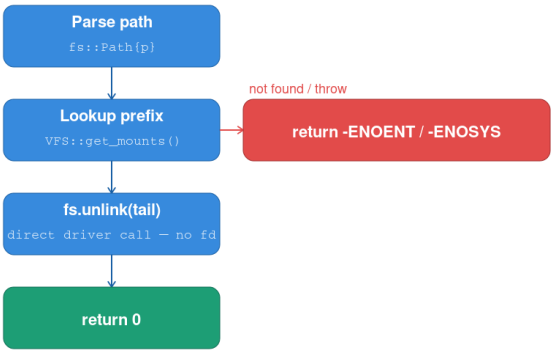

Figure 4.4: `sys_unlink`: same prefix lookup as `sys_open`, direct driver call, no fd lifecycle.

4.2 Virtio PCI Layer

The constructor validates vendor ID (`PCI::VENDOR_VIRTIO`) and product ID — 0x1040–0x107F for modern devices; 0x1000–0x103F legacy devices are rejected. Capability discovery follows `cap_next` links checking for `PCI_CAP_ID_VNDR`; both 32-bit and 64-bit BARs are handled. Three regions are recorded: `virtio_pci_common_cfg`, device-specific configuration, and the notification region with its `notify_off_multiplier`.

The driver resets the device, sets `ACKNOWLEDGE` and `DRIVER`, reads the 64-bit feature word in two 32-bit halves, unconditionally requires `VIRTIO_F_VERSION_1`, optionally accepts `VIRTIO_F_INDIRECT_DESC` and `VIRTIO_F_EVENT_IDX`, writes back the negotiated set, sets `FEATURES_OK`, and re-reads `device_status` to confirm acceptance.

`Split_queue` performs a significant amount of control-plane register work in its constructor. On construction it writes to `queue_select` to target the correct per-queue register set in `virtio_pci_common_cfg`, reads back `queue_size`, computes and stores the notification address from `queue_notify_off` and `notify_off_multiplier`, assigns `queue_msix_vector` and suppresses config interrupts by writing `VIRTIO_MSI_NO_VECTOR` to `config_msix_vector`, writes the physical addresses of the descriptor table, available ring, and used ring to `queue_desc`, `queue_driver`, and `queue_device` respectively, and finally sets `queue_enable = 1` to activate the queue. After con-

struction, no further `virtio_pci_common_cfg` accesses occur — all subsequent I/O proceeds through pre-computed pointers to the three ring structures and the notification register.

The three ring structures are allocated with the alignment requirements mandated by the specification: 16 bytes for the descriptor table, 2 bytes for the available ring, and 4 bytes for the used ring. The per-queue notification address is computed as:

$$notify_addr = notify_region_base + queue_notify_off \times notify_off_multiplier$$

The free-descriptor pool is a `std::vector` of indices pre-allocated with `reserve(_QUEUE_SIZE)` and initialized to [0, queue_size), providing O(1) allocation (pop from back) and reclamation (push to back) with no dynamic memory allocation on the I/O path.

Enqueue accepts a `VirtTokens` vector. Each `VirtToken` pairs a `VirtBuffer` — a `std::span<uint8_t>` over a guest-physical buffer — with a 16-bit flags word. One descriptor is allocated per token; all but the last have `VIRTQ_DESC_F_NEXT` ORed into their flags and their `next` field filled with the following descriptor's index, forming a chain. The head index is placed into the available ring at position `_avail_ring->idx & (queue_size - 1)`. A `memory_order_release` fence is then issued before incrementing `_avail_ring->idx`, guaranteeing all descriptor writes are globally visible before the device observes the updated index. A subsequent `memory_order_seq_cst` fence precedes the check of `_used_ring->flags` for notification suppression (`VIRTQ_USED_F_NOTIFY`); if the device has not suppressed kicks, `_notify_device()` writes the queue index as a 16-bit value to the pre-computed notification register address.

Dequeue reads one `virtq_used_elem` at position `_last_used_idx & (queue_size - 1)`, optionally returning the `len` field to the caller via an output pointer to indicate how many bytes the device wrote into writable buffers. It then walks the descriptor chain from `used_elem.id`, reconstructing one `VirtToken` per descriptor and returning each index to the free pool, until a descriptor without `VIRTQ_DESC_F_NEXT` is reached. Whether a completed entry is available can be checked in advance with `has_processed_used()`, which returns true when `_last_used_idx == _used_ring->idx` — the `volatile` qualifier on `_used_ring->idx` ensures the comparison always reads the device-updated value from memory.

Notification suppression is controlled by two inline methods. `suppress()` writes `VIRTQ_AVAIL_F_NO_INTERRUPT` into the available ring's `flags` field, requesting

that the device refrain from sending completion interrupts. `unsuppress()` clears this by writing `VIRTQ_AVAIL_F_INTERRUPT`, re-enabling interrupt delivery.

`enable_msix(uint8_t required_msix_count)` activates MSI-X interrupt delivery for the device. When `required_msix_count` is greater than zero, the method sets the internal `_msix_enabled` flag and calls `_pcidev.probe_resources()` followed by `_pcidev.parse_capabilities()` to enumerate PCI BARs and locate the MSI-X capability structure. It asserts that `msix_cap() > 0`, meaning the device advertises MSI-X support, then calls `_pcidev.init_msix()` to write the MSI-X enable bit in the capability header and `get_msix_vectors()` to read back the number of vectors the device provides. A further assertion verifies that the available vector count meets the requested minimum. The return value is the actual vector count, which the caller may use to bind individual queues. On the `Split_queue` side, `setup_msix_vector(queue_idx, vector)` writes the allocated vector number into `queue_msix_vector` inside `virtio_pci_common_cfg`, binding a specific MSI-X interrupt line to that queue's completion events. The driver pairs this with `Events::get().subscribe(callback)`, which registers a callback in the IncludeOS event system and returns an `event_num`; passing `IRQ_BASE + event_num` as the vector argument connects the hardware interrupt directly to the callback without polling.

4.3 VirtioFS Device Driver

4.3.1 Initialisation

All members are constructed via member-initializer syntax. `Virtio_control`'s constructor completes PCI negotiation before returning; `set_driver_ok_bit` is called immediately. The `FUSE_INIT` request is submitted as a two-token chain (readable request | writable response buffer); the driver spins on `has_processed_used`. The daemon's major version must match exactly; its minor must meet or exceed the driver's minimum. `max_write` is stored reduced by `FUSE_BUFFER_HEADER_SIZE` (4096 bytes). Finally an `fs::Filesystem` struct is registered under `VirtioFS{id}`.

4.3.2 Inode Resolution

`_lookup_inode` sends a three-token `FUSE_LOOKUP` chain: request header (readable) | null-terminated path (readable) | response buffer (writable). The returned inode number, file handle, and seek offset are stored in `_fd_info_map` keyed by the

POSIX descriptor integer, eliminating repeated lookups for an open file.

4.3.3 Open, Create, and Close

`_open_exist` calls `_lookup_inode` then issues `FUSE_OPEN`, storing the file handle with initial seek offset zero. `_open_creat` issues `FUSE_CREATE` against the root inode, combining lookup and open in one round-trip. `close` sends `FUSE_RELEASE` on the request queue, removes the descriptor from `_fd_info_map`, then sends `FUSE_FORGET` on the high-priority queue to promptly decrement the daemon's inode reference count.

4.3.4 Read and Write

All read and write operations deliver caller buffers directly to VirtioFSD via Virtio descriptor chains with no intermediate copy.

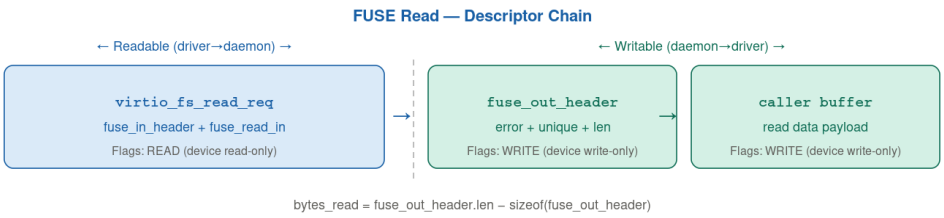

Figure 4.5: FUSE read chain: request header (readable) | response header (writable) | caller buffer (writable). Bytes read = `fuse_out_header.len` − `sizeof(fuse_out_header)`.

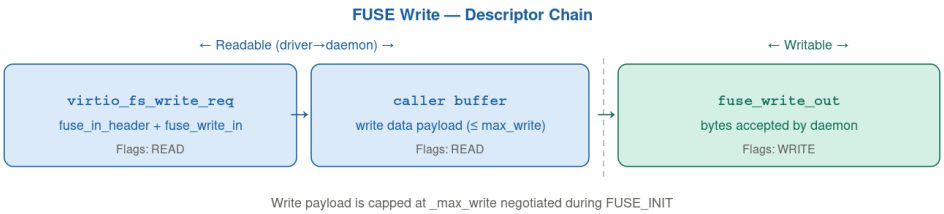

Figure 4.6: FUSE write chain: request header (readable) | caller buffer (readable, capped at `_max_write`) | `fuse_write_out` (writable).

The scatter-gather variants `readv` and `writev` append one `VirtToken` per `iovec` element, exploiting descriptor chaining to fill or drain discontiguous caller buffers with no aggregation buffer.

4.3.5 Seek, Unlink, and Registration

`lseek` updates `_fd_info_map`'s offset for `SEEK_SET` and `SEEK_CUR` without issuing any Virtio operations — all offset state is authoritative in the driver. `unlink` issues `FUSE_UNLINK` with the path pointer offset by one byte to strip the leading separator added by the VFS layer.

Chapter 5

Evaluation

This chapter presents the experimental evaluation of the VirtioFS implementation for IncludeOS. Three research questions are investigated through a set of controlled microbenchmarks and an end-to-end application workload. Section 5.1 describes the common infrastructure shared by all experiments. Sections 5.2–5.4 then present the setup, results, and analysis for each research question in turn.

5.1 Experimental Setup

5.1.1 Virtual-Machine Infrastructure

All experiments are executed inside virtual machines managed by QEMU/KVM on the same host workstation, ensuring that CPU microarchitecture and memory topology are constant across every run. A shared host directory is exposed to the guest through VirtioFS, the primary transport under evaluation. The host directory itself is mounted on a **tmpfs** ramdisk so that host-side storage latency and caching effects are eliminated from the measurements. The VirtioFS daemon is launched with direct I/O passthrough (—**allow-direct-io**) for guest and daemon caching disabled (—**cache never**).

IncludeOS machines run with a single virtual processor and 3 gibibytes of ram. Benchmark code and IncludeOS are linked and bundled into a bootable image. The experiments are executed with two VirtioFS driver configurations:

Polling. After submitting a request to the VirtioFS virtqueue, the IncludeOS driver busy-polls the used-ring until the host returns a completion. No interrupt is taken and the vCPU never halts.

Listing 5.1: Busy-polling

```
_req.enqueue(tokens);
while(_req.has_processed_used()); // Busy-polling
_req.dequeue();
```

IRQ (MSI-X). After submission, the vCPU executes a `HLT` instruction and sleeps until a MSI-X interrupt from the VirtioFS device signals completion. We register a dummy function as the deferred callback routine for the request virtqueue. Using interrupts frees the virtual processor for other work, reduces energy and CPU utilization, but introduces extra latency compared to busy-polling.

Listing 5.2: VirtioFS deferred interrupt setup

```
// VirtioFS init function
_control.enable_msix(1);
auto event_num = Events::get().subscribe(
  {this, &VirtioFS_device::_fuse_log_reply}
);
d.setup_msix_vector(0, IRQ_BASE + event_num);
_control.set_driver_ok_bit();

// Enable_msix function
uint8_t Virtio_control::enable_msix(uint8_t required_msix_count) {
  if (required_msix_count > 0) {
    _msix_enabled = true;

    /* Grabbing PCI resources. Dependency for MSIX code */
    _pcidev.probe_resources();
    _pcidev.parse_capabilities();

    /* Checking for the availablity of MSIX capability */
    bool supports_msix = _pcidev.msix_cap() > 0;
    CHECK(supports_msix, "Device␣supports␣MSIX!");
    _virtio_panic(supports_msix);

    /* Driver requires required_msix_count # of vectors */
    _pcidev.init_msix();
    uint8_t msix_vector_count = _pcidev.get_msix_vectors();
    bool sufficient_msix_count = msix_vector_count >= required_msix_count
    CHECK(sufficient_msix_count, "Sufficient␣msix␣vector␣count␣(%d)", msi
```

```
    _virtio_panic(sufficient_msix_count);

    return msix_vector_count;
  }

// Split queue setting the MSI-X vector
cfg.queue_msix_vector = VIRTIO_MSI_NO_VECTOR;
```

Linux machines execute with four virtual processors and 8 gibibytes of ram. The machines run with more resources to reduce the scheduling and memory pressure from other processes running on the system. The benchmark processes run with minimal interference.

5.1.2 Timing

All benchmarks use the same **chrono::high_resolution_clock** clock in C++ for timestamps. The benchmarks calculate the time delta before and after the function to be timed.

5.1.3 Reproducible Builds

All benchmark programs are compiled with the Nix package manager to guarantee bit-for-bit reproducibility across machines.

IncludeOS unikernel binaries are built with `includeos.stdenv.mkDerivation`; the resulting `virtiofs_bench.elf.bin` image is passed directly to the `boot` launcher. Linux binaries are compiled with the `pkgsMusl` static C library toolchain to minimize runtime dependencies and prevent dynamic linker behavior from confounding results.

5.1.4 Host machine

The machine is `Lenovo LOQ 15AHP10` laptop running a `AMD Ryzen 7 250 (8 cores and 16 threads)` with 32GiB of RAM. The host operating system is **Debian 13 (Trixie)** with the kernel version **6.12.63**.

5.1.5 POSIX I/O Benchmark (Experiments 1 & 2)

The synchronous-I/O benchmark exercises VirtioFS through the standard POSIX file API (`open/read/write/lseek/close`). In IncludeOS this translates directly

into driver calls with no kernel syscall boundary; in Linux the calls traverse the VFS and protections mechanims before reaching the `virtiofsd`-backed driver.

A 200 MiB binary file (`syncio_testing_file.bin`) pre-placed on the shared `tmpfs` serves as the read source. Write benchmarks create a fresh output file on each run and remove it afterwards. Files are opened with `O_DIRECT` to bypass the page cache.

The benchmark iterates over chunk sizes from 8 KiB to 248 KiB in steps of 8 KiB (31 sizes). For each chunk size 30 independent runs are executed. Four access patterns are measured: *sequential read*, *sequential write*, *random read*, and *random write*. The random benchmarks permute chunk indices with a fixed seed (Mersenne Twister, seed 42).

Each run calculates the time delta of the innermost processing loop. The result is recorded as elapsed time in milliseconds and as throughput in mebibytes per second (MiB/s), computed as filesize/elapsed.

5.1.6 io_uring Benchmark (Experiment 3)

The io_uring benchmark is equivalent to POSIX with applied io_uring patches. The submission queue depth is set to 1; a single outstanding I/O is submitted and waited upon before the next is issued, matching the serialized access pattern of the POSIX benchmark. io_uring is configured with a polling kernel thread using `IORING_SETUP_SQPOLL`. This spawns a kernel thread that polls the submission queue without requiring a `io_uring_enter` system call from the application. This benchmark runs only on Linux.

5.1.7 C63 Video Decoder Benchmark

The c63 codec is a simplified video codec developed for educational purposes. C63 uses buffered file I/O that calls vectorized I/O.

The decoder `c63dec` reads a compressed `.c63` file from the VirtioFS-mounted directory and writes the decoded raw YUV frames back to the same directory, exercising both the read and write paths of the VirtioFS driver under a realistic, compute-bound workload.

The input file is a 60-frame encoded of West Wind Easy (`wwe.c63`). The entire decode-and-write sequence is timed with `std::chrono::high_resolution_clock` and repeated 30 times per system configuration. Run 0 is treated as a cold-start

observation and excluded from statistical analysis; runs 1–29 are used to compute means and confidence intervals.

Three system configurations are compared: Linux, IncludeOS Polling, and IncludeOS IRQ.

5.2 RQ1 – VirtioFS Throughput: IncludeOS vs. Linux

RQ1: *Can a VirtioFS implementation in IncludeOS have a higher I/O throughput than Linux?*

5.2.1 Results

Figure 5.1 shows the mean throughput (with 95% confidence intervals as shaded bands) across all 31 chunk sizes for the four access patterns, comparing Linux POSIX, IncludeOS Polling, and IncludeOS IRQ. Figure **??** summarises peak throughput as a bar chart.

IncludeOS Polling vs. Linux POSIX. IncludeOS Polling achieves the highest throughput across every access pattern and chunk size. At peak (248 KiB chunks for reads, around 192–248 KiB for writes), it sustains **4,786 MiB/s** for sequential reads and **2,076 MiB/s** for sequential writes — representing speedups of $\times$**1.74** and $\times$**1.84** over the Linux POSIX baseline respectively. Random access exhibits nearly identical relative gains: $\times$1.78 for random reads and $\times$1.83 for random writes.

The advantage is even more pronounced at small chunk sizes, where system call and interrupt overhead dominate. At 8 KiB chunks, IncludeOS Polling delivers **804 MiB/s** for sequential reads against only **266 MiB/s** for Linux POSIX — a $\times$**3.02** speedup. This highlights how the elimination of the kernel-mode/user-mode transition, the VFS dispatch, and the FUSE/`virtiofsd` round-trip dramatically reduces per-request latency when requests are frequent and small.

IncludeOS IRQ vs. Linux POSIX. IncludeOS IRQ outperforms Linux POSIX across the entire chunk range, but by a narrower margin than polling. Peak sequential read throughput reaches **3,454 MiB/s** ($\times$1.26 over Linux POSIX), and peak sequential write reaches **1,696 MiB/s** ($\times$1.50). At 8 KiB chunks the IRQ mode achieves $\times$1.24 over Linux for sequential reads.

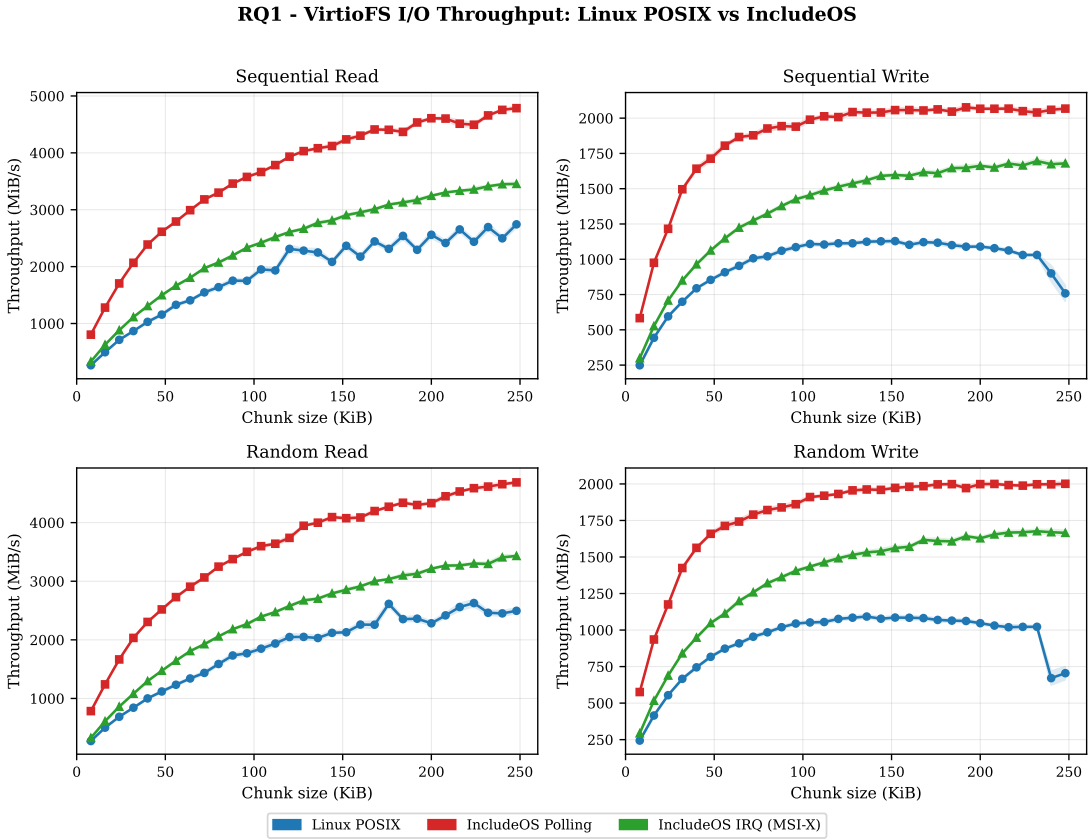

Figure 5.1: RQ1 – Mean VirtioFS throughput (MiB/s) vs. chunk size for all four access patterns. Shaded bands represent 95% confidence intervals over 30 runs.

The performance gap between IRQ and Polling within IncludeOS is itself instructive. Polling outperforms IRQ by ×1.39 at peak sequential read and ×1.22 at peak sequential write. This gap is attributable to the round-trip cost of MSI-X delivery: after every `HLT`, the vCPU must be woken from its halted state by the hypervisor before execution can resume, a path that involves an exit from the guest, interrupt injection, and a re-entry.

Throughput scaling with chunk size. All three systems exhibit a consistent pattern: throughput rises steeply from 8 KiB and plateaus beyond approximately 128–192 KiB, after which there is little additional gain from larger transfers. This plateau reflects the saturation of the underlying `tmpfs` memory bandwidth. The plateau for IncludeOS Polling, however, sits roughly twice as high as that for Linux POSIX.

5.2.2 Analysis

The throughput advantage of IncludeOS over Linux can be attributed to three structural differences:

1. **No syscall overhead.** In IncludeOS, a call to `read()` or `write()` dispatches directly to the VirtioFS driver without a privilege-level switch and additional layers.
2. **Absence of OS interference.** As a unikernel, IncludeOS runs a single cooperative task. There is no scheduler preemption overhead during the benchmark.

Answer to RQ1: Yes. The IncludeOS VirtioFS implementation achieves substantially higher I/O throughput than Linux across all measured access patterns, chunk sizes, and notification modes. In polling mode the peak speedup reaches ×1.74–1.84 at large chunk sizes and ×2.34–3.02 at 8 KiB.

5.3 RQ2 – Tweaking Linux: io_uring SQPOLL

RQ2: *Why is the I/O throughput in VirtioFS Linux slower than IncludeOS? Can we tweak Linux to achieve equivalent performance?*

5.3.1 Results

Figure 5.2 compares Linux POSIX and Linux io_uring SQPOLL across the four access patterns. Figure 5.3 shows the per-chunk-size speedup ratio of io_uring relative to POSIX.

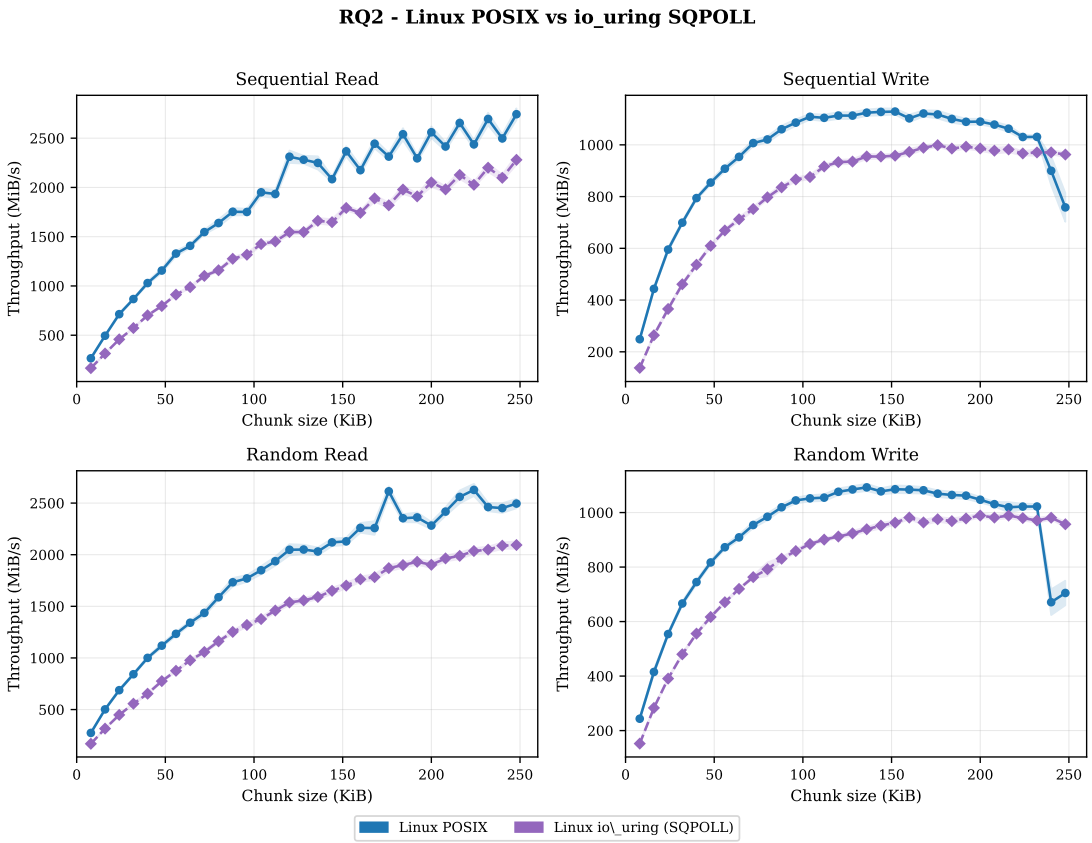

Figure 5.2: RQ2 – Mean VirtioFS throughput: Linux POSIX vs. Linux io_uring SQPOLL. Shaded bands are 95% confidence intervals.

Contrary to expectations, io_uring SQPOLL is *slower* than Linux POSIX across the entire chunk-size range and all four access patterns. Peak sequential read throughput with io_uring reaches only **2,280 MiB/s** versus **2,743 MiB/s** for POSIX (a regression of ×0.83). For sequential writes the regression is ×0.88. At the smallest chunk size of 8 KiB, the penalty is even larger: io_uring achieves only **166 MiB/s** for sequential reads, against **266 MiB/s** for POSIX (×0.62).

Figure 5.4 places all four configurations side by side to contextualise these findings relative to the IncludeOS baselines.

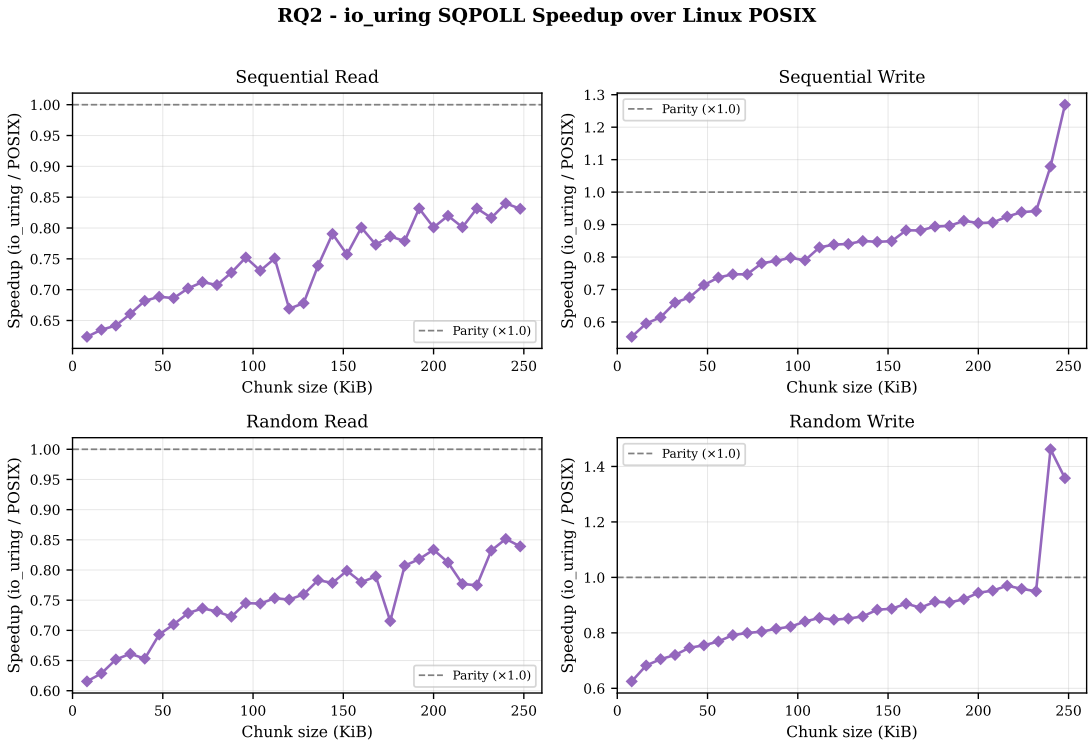

Figure 5.3: RQ2 – io_uring SQPOLL throughput as a ratio of Linux POSIX throughput (values above 1.0 indicate an improvement, below 1.0 a regression). The dashed grey line marks parity.

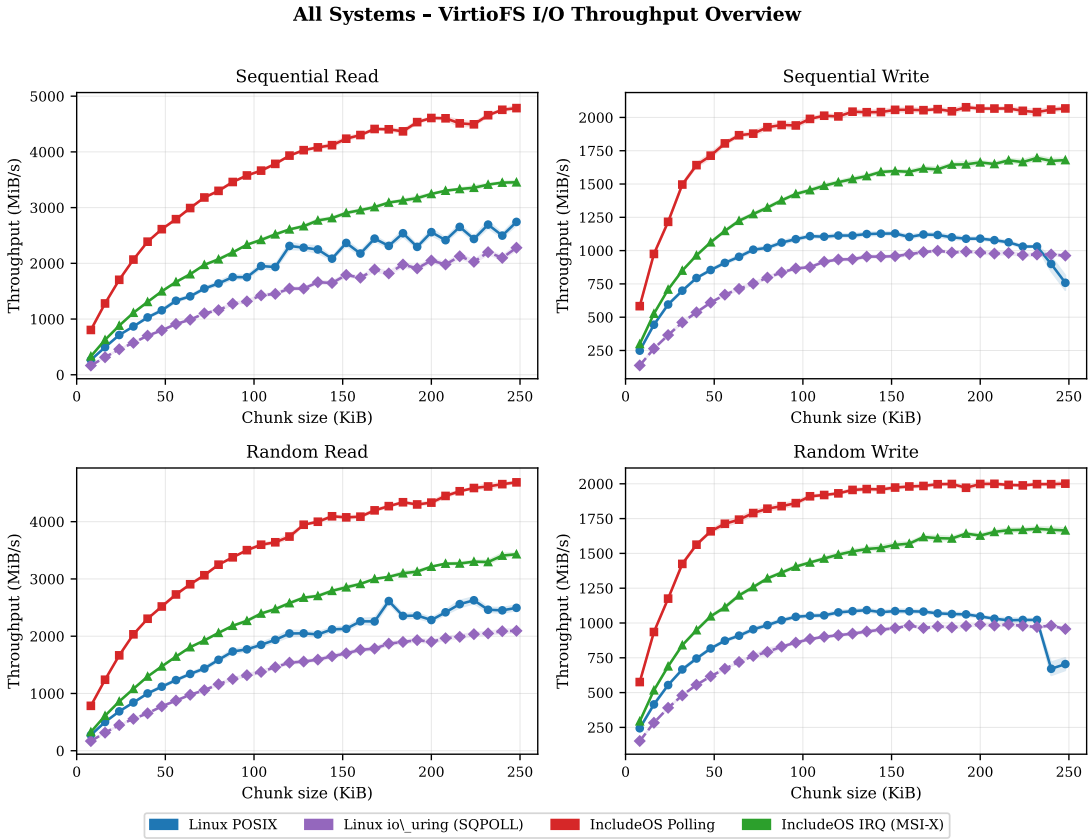

Figure 5.4: All system configurations – VirtioFS throughput overview for all four access patterns. io_uring falls below even the Linux POSIX baseline.

5.3.2 Analysis

Queue depth of 1. The benchmark deliberately maintains a single outstanding I/O at a time. SQPOLL provides its greatest benefit at high queue depths where many I/O operations can be batched into the submission ring with a single kernel notification. At depth 1 the overhead of ring management is incurred for every single operation without amortization.

Answer to RQ2: The throughput gap between Linux and IncludeOS is caused by the multiple software layers interposed between the application and the VirtioFS virtqueue: system calls, the VFS, FUSE framing, and the `virtiofsd` user-space daemon. Replacing synchronous POSIX calls with io_uring SQPOLL does not close this gap; on the contrary, at queue depth 1 with VirtioFS, SQPOLL introduces a throughput regression of 9–38% compared with ordinary POSIX I/O.

5.4 RQ3 – Impact on Video Decoding

RQ3: *What impact will enhanced I/O throughput have on common tasks such as video decoding?*

5.4.1 Results

Figure 5.5 shows a box plot of decode times (runs 1–29) for the three system configurations.

Table 5.1: c63dec benchmark statistics (runs 1–29; run 0 excluded as cold start). Speedup is relative to Linux POSIX.

Configuration	Mean (s)	Std (s)	Min (s)	Max (s)	Speedup
Linux POSIX	4.861	0.032	4.820	4.957	×1.00
IncludeOS Polling	3.759	0.015	3.731	3.790	×**1.29**
IncludeOS IRQ (MSI-X)	4.096	0.030	4.020	4.144	×1.19

IncludeOS Polling. IncludeOS Polling achieves a mean decode time of **3.759 s**, compared with **4.861 s** for Linux POSIX — a speedup of ×**1.29**, or a **22.7% reduction** in wall-clock decode time.

IncludeOS IRQ. IncludeOS IRQ reduces decode time to **4.096 s**, a ×**1.19** speedup (18.6% reduction) over Linux. The 8.5% gap between IRQ and Polling within IncludeOS reflects the interrupt-delivery overhead noted in Section 5.2: for

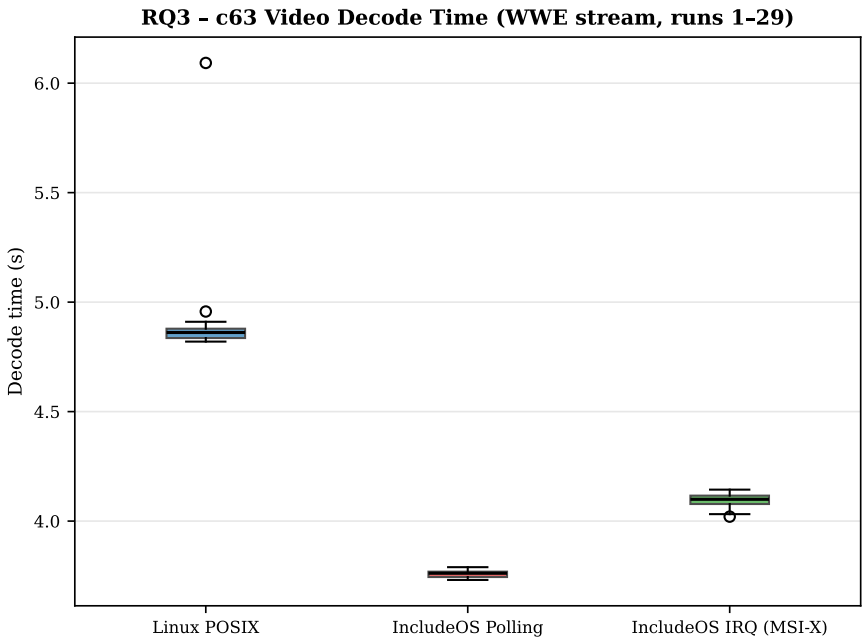

Figure 5.5: RQ3 – Distribution of c63 decode times over 29 warm runs. Boxes span the interquartile range; whiskers extend to the most extreme non-outlier observation.

a codec that issues many small I/O requests per decoded frame, the accumulated cost of MSI-X wakeup cycles is significant.

Cold-start behaviour. Run 0 reveals a striking asymmetry between Linux and IncludeOS. Linux requires **6.092 s** on the cold run — a **25.3%** penalty over its warm mean — caused by page-fault handling, dynamic loader initialisation, and I/O scheduling setup. IncludeOS Polling's cold run takes **3.772 s**, only **0.4%** above its warm mean; IncludeOS IRQ shows a similarly negligible **0.2%** cold overhead. This behaviour reflects the unikernel's single-process, pre-linked execution model: there is no dynamic linker, no kernel process initialisation, and no page-table population at startup.

5.4.2 Analysis

The magnitude of the end-to-end speedup (22.7%) is smaller than the raw peak-throughput speedup seen in the microbenchmarks (×1.74 for sequential reads). This is expected: the c63dec workload is not I/O-bound. A substantial fraction of execution time is spent in the CPU decode pipeline (VLC decoding, IDCT, motion compensation), which is identical across all configurations. The measured speedup therefore represents the I/O contribution to the total workload, scaled by

the Amdahl factor of the non-I/O fraction.

Answer to RQ3: The higher I/O throughput of the IncludeOS VirtioFS implementation translates directly into faster end-to-end execution of a realistic video decoding task. IncludeOS Polling completes decoding **22.7% faster** than Linux POSIX. The unikernel also eliminates the significant cold-start penalty observed on Linux, making every execution faster and more predictable.

5.5 Threats to Validity

Internal validity. The `tmpfs` shared directory eliminates disk I/O from the measurements but also places an upper bound on achievable throughput set by the host's DRAM bandwidth. Both Linux and IncludeOS are subject to the same bound; the relative comparison remains valid, but absolute numbers cannot be directly extrapolated to persistent storage scenarios.

External validity. The benchmark uses a single test file size (204.8 MiB) and a fixed random seed. Workloads with smaller files, varying access patterns, or concurrency (multiple I/O threads) may exhibit different relative performance. The c63dec workload is a low-resolution educational codec; production video workloads with larger frames, higher frame rates, or compressed formats (H.264, AV1) may be more or less I/O-bound.

Reproducibility. All experiments use Nix-pinned dependency graphs; the build and runtime environments are fully reproducible. Nevertheless, results may differ on hardware with different DRAM bandwidth, CPU microarchitecture, host or KVM implementation.

Chapter 6

Conclusion and Future Work

This thesis has presented the design, implementation, and evaluation of a VirtioFS driver for the IncludeOS unikernel. The evaluation addressed three research questions through a suite of controlled microbenchmarks and an end-to-end video decoding workload.

Regarding **RQ1**, the results confirm that IncludeOS can achieve significantly higher VirtioFS I/O throughput than a standard Linux guest. In polling mode, peak throughput exceeds Linux POSIX by ×1.74–1.84 across sequential and random patterns, and by up to ×3.02 at small chunk sizes where per-request overhead dominates. Even the interrupt-driven MSI-X mode outperforms Linux by ×1.26–1.53. The advantage stems from the structural simplicity of the unikernel: the absence of a kernel boundary, simple VFS dispatch and no permission checking.

Regarding **RQ2**, the investigation into Linux-side optimisations via io_uring SQPOLL produced a counterintuitive result: SQPOLL *degrades* VirtioFS throughput by 9–38% relative to ordinary POSIX I/O. The cause of the regression is not known, and must be investigated further.

Regarding **RQ3**, the end-to-end c63 video decoder benchmark shows that the I/O throughput advantage of IncludeOS translates into meaningful application-level gains. IncludeOS Polling completes decoding 22.7% faster than Linux. Notably, IncludeOS completely eliminates the 25.3% cold-start penalty observed on Linux, making unikernel execution faster from the very first run.

Taken together, these findings validate the motivation for implementing VirtioFS in a unikernel: the removal of general-purpose OS abstractions that are unnecessary in a single-application environment yields substantial, measurable I/O performance

improvements that propagate to realistic application workloads.

6.1 Future Work

Several directions remain open for investigation:

- **Higher io_uring queue depths.** The RQ2 benchmark deliberately used queue depth 1 to match the serialized POSIX baseline. Evaluating io_uring at depths of 4, 8, 32, or higher could reveal whether concurrent submission changes the picture.
- **Persistent storage backends.** All benchmarks in this thesis used a `tmpfs` ramdisk on the host to eliminate disk latency from the measurements. Repeating the evaluation with an NVMe SSD backend would reveal whether the driver-level performance advantage is preserved under disk-latency constraints, and whether the FUSE round-trip cost is dominated by device latency in that regime.
- **Production codec workloads.** The c63 decoder is an educational tool with limited resolution and frame rate. Benchmarking with production codecs such as FFmpeg decoding H.264 or AV1 content would provide more valid evidence of the application-level impact.

Bibliography

[1] G. J. Popek and R. P. Goldberg, "Formal requirements for virtualizable third generation architectures," *Commun. ACM*, vol. 17, no. 7, pp. 412–421, Jul. 1974.

[2] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX ATC*, Anaheim, CA, 2005. `https://www.qemu.org`

[3] A. Kivity et al., "KVM: the Linux virtual machine monitor," in *Proc. Linux Symposium*, Ottawa, vol. 1, 2007, pp. 225–230.

[4] KVM project, "ioeventfd," Linux kernel documentation, 2024. `https://www.kernel.org/doc/html/latest/virt/kvm/api.html`

[5] QEMU project, "Vhost-user protocol specification," 2024. `https://qemu-project.gitlab.io/qemu/interop/vhost-user.html`